Day 3 Evidence Workbooklet — Instructor Key

Types, Casting, and Truthiness

Engineering practice → data type → safe conversion → truthiness check → support route

Instructor key. Same layout as student copy; solutions filled in response boxes. Print format: duplex, left-stapled packet.

Name		**Section**		**Date**	
Score	_____/40		Mastery check	secure / developing / needs support	

Your team is continuing the pump-flow record from Day 2. Today the values arrive from an intake form before they are used in a notebook. Some entries arrive as text, some are already numbers, and some fields are blank. Before the team can make safe threshold decisions tomorrow, the record has to distinguish type, conversion, and truthiness.

The problem today is not branch routing. The problem is whether a value is text, a number, or Boolean-like evidence, and whether a conversion or truthiness check is safe to trust.

Engineering task	Prepare a pump-flow intake record so later threshold and go/no-go decisions are based on converted values rather than misleading text labels.
Computational move	Classify values by type, cast text when safe, and interpret truthiness without confusing a non-empty label for an approved result.
Python focus	`str`, `float`, `int`, `bool`, `type(...)`, `float(...)`, empty string, non-empty string, zero, and nonzero values.
Evidence to keep	original value, Python type, cast expression, cast result, truthiness result, and one corrected intake-note sentence.

Day 3 type and truthiness rules

1. Text that looks like a number is still text until code converts it. Example: `"18.6"` is a string, while `18.6` is a float.
2. A cast such as `float("18.6")` produces a numeric value, but it does not change the original text variable unless an assignment stores the result.
3. Empty strings are falsey. Non-empty strings are truthy, even when the text says `"no"` or `"0"`.
4. Numeric zero is falsey. Most nonzero numbers are truthy.
5. Truthiness is evidence about empty/non-empty or zero/nonzero status; it is not the same as engineering approval.

1 Read the intake record as typed data

recognize predict

Use the intake record below. First identify what kind of value each line stores. Do not make a go/no-go decision yet.

```
# Intake form values from the pump station
pump_id = "P-22"
target_flow_text = "20.0"
measured_flow_text = "18.6"
operator_note = "needs review"
approved_text = "no"
spare_comment = ""
retry_count = 0
sensor_ready = True

# Safe conversions for later calculations
target_flow_lpm = float(target_flow_text)
measured_flow_lpm = float(measured_flow_text)
gap_lpm = target_flow_lpm - measured_flow_lpm
```

Pts.	Prompt	My evidence
1	Which two variables store numbers as text before conversion?	**Key:** `target_flow_text` and `measured_flow_text`.
1	Which variable stores an empty string?	**Key:** `spare_comment` stores `""`.
1	Which variable stores a whole-number count rather than text?	**Key:** `retry_count`.
1	Which variable already stores a Boolean value?	**Key:** `sensor_ready` stores `True`.

2 Classify type before using a value

recognize explain

Complete the type table. Use Python type names such as `str`, `int`, `float`, and `bool`. Then write why the type matters for later work.

Pts.	Variable or expression	Type	Why this matters before a threshold check
2	`target_flow_text`	**Key:** `str`	**Key:** Original text `"20.0"`; not safe as a numeric threshold until cast.
2	`target_flow_lpm`	**Key:** `float`	**Key:** Converted numeric value `20.0` for calculations.
2	`approved_text`	**Key:** `str`	**Key:** Stores label `"no"`; truthiness is not approval.
2	`sensor_ready`	**Key:** `bool`	**Key:** Already stores Boolean evidence.

Common trap to check

The text `"20.0"` may look like a number, but Python will treat it as text until it is converted. A reliable engineering record names both the original field and the converted field.

3 Cast text to numbers without rewriting the source record

predict trace

A cast produces a new value. It changes a stored variable only if the cast result is assigned to a name. Complete the table.

Pts.	Expression or assignment	Value produced or stored	What changed?
2	`float(target_flow_text)`	**Key:** `20.0`	**Key:** Produces a float; original text variable is unchanged unless assigned.
2	`float(measured_flow_text)`	**Key:** `18.6`	**Key:** Produces a float from the measured-flow text.
2	`target_flow_lpm = float(target_flow_text)`	**Key:** `20.0`	**Key:** Stores the converted float in a new numeric field.
2	`gap_lpm = target_flow_lpm - measured_flow_lpm`	**Key:** `1.4`	**Key:** Uses converted numeric fields: `20.0 - 18.6`.

Pts.	Prompt	My response
2	Why should the notebook keep the original text field and the converted numeric field separate?	**Key:** The text field preserves the source record; the numeric field is safe for calculations. Keeping both supports audit and debugging.
2	What would be unsafe about calculating with `target_flow_text` as if it were already a number?	**Key:** It is a string, so arithmetic may fail or act like text rather than numeric calculation. Cast first.

4 Interpret truthiness without mistaking labels for approval

predict explain

Python truthiness can be useful, but it can also mislead an engineering record when text labels are treated like decisions. Complete the table.

Pts.	Value or variable	Truthy or falsey?	Engineering interpretation
1	`spare_comment` storing `""`	**Key:** falsey	**Key:** Empty string; no comment text.
1	`operator_note` storing `"needs review"`	**Key:** truthy	**Key:** Non-empty string; still a review note, not approval.
1	`approved_text` storing `"no"`	**Key:** truthy	**Key:** Non-empty string, but text means not approved.
1	`retry_count` storing `0`	**Key:** falsey	**Key:** Numeric zero is falsey.
1	`sensor_ready` storing `True`	**Key:** truthy	**Key:** Actual Boolean True.

Truthiness warning

`bool("no")` is `True` because the string is not empty. That does not mean the operator approved the record.

Pts.	Prompt	My response
2	In one sentence, explain why truthiness is not the same as engineering approval.	**Key:** A non-empty string such as `"no"` is truthy, but the actual text still means not approved.

5 Debug a type or truthiness mistake

debug test explain

A teammate writes this intake note before tomorrow's threshold check:

Intake note to debug

The record is approved because `approved_text` has a value, and the flow gap can be calculated directly from the two text fields.

Pts.	Prompt	My response
2	What part of the teammate's note confuses truthiness with approval?	**Key:** It treats `approved_text` having any non-empty value as approval, even though the value is `"no"`.
2	What part of the note confuses text values with numeric values?	**Key:** It says the flow gap can be calculated directly from the text fields; they should be cast to floats first.
2	Write one non-mutating Python expression that checks the type of `approved_text`.	**Key:** `type(approved_text)`.
2	Rewrite the intake note so it separates text fields, converted numeric fields, and actual approval evidence.	**Key:** The intake stores text fields such as `approved_text = "no"` and numeric fields such as `target_flow_lpm = 20.0` after casting. Approval should be checked explicitly from the approval value/rule, not from truthiness alone.

Bridge to Day 4

Tomorrow's boundary checks will use comparison operators such as `<`, `<=`, and `==`. Those checks are safer when today's type evidence is already clear: converted numbers for numeric thresholds, and explicit labels or Boolean values for decision evidence.

6 Mastery check and support route

transfer explain

Use your own evidence from this workbooklet. Do not write a generic answer.

Pts.	Prompt	My response
2	Give one example where text looked like a number but needed a cast before safe calculation.	**Key:** `"20.0"` in `target_flow_text` must be cast with `float(...)` before numeric threshold work.
2	Give one example where truthiness could mislead an engineering record if the team did not read the actual value.	**Key:** `bool("no")` is `True` because the string is non-empty, but the label says no.

My mastery check

Mark the strongest statement that is true after this workbooklet.

☐	Secure: I can classify a value's type, predict a safe cast, and explain why truthiness is not the same as approval.
☐	Developing: I can identify some types or casts, but I still need support with string truthiness or converted fields.
☐	Needs support: I need a smaller example before I can use type and truthiness evidence in a record.

My next support move

My issue is mostly: setup / Python syntax / tracing / type conversion / truthiness / confidence / time.
The first value where I got uncertain was: _____
My next small action is: ask a partner / ask instructor / rebuild the type table / rerun one expression / try the recovery version.

Workbooklet target: I can classify Python values, cast text safely, interpret truthiness, and prepare typed evidence before threshold decisions.